Lab 5: Static Code Analysis

Objective

To enhance Python code quality, security, and style by utilizing **static analysis tools** (Pylint, Bandit, and Flake8) to detect and rectify common programming issues in the python script assigned to you.

Duration:

60 minutes

Software Requirements

- **Setup python virtual env**
  - **python3 -m venv venv**
  - **source venv/bin/activate**
- **Install Pylint**
  - **pip install pylint**
- **Install Bandit**
  - **pip install bandit**
- **Install Flake8**
  - **pip install flake8**
- **Download the Python script from the GitRepo assigned to you.**

Learning Outcomes

By the end of this lab, you will be able to:

- **Understand the purpose and scope of static code analysis.**
- **Identify and resolve common Python programming issues**, including mutable default arguments, overly broad exceptions, and insecure coding patterns.
- **Effectively use Pylint, Bandit, and Flake8** to analyze Python code.
- **Interpret static analysis reports** and apply fixes based on issue severity.
- **Reflect on the contribution of static analysis** to overall code quality and maintainability.

Introduction

In modern software development, ensuring code quality, security, and maintainability is paramount. **Static code analysis** is a crucial technique that involves examining source code without executing it to find potential bugs, vulnerabilities, and style violations. This lab will guide you through using three industry-standard Python static analysis tools:

- **Pylint**: Your "strict code reviewer" for **code quality and logical errors**. It flags issues like unused variables, poor practices, and potential design flaws.
- **Flake8**: Your "grammar checker" for Python code, enforcing **PEP 8 style guidelines**. It combines checks for formatting, whitespace, line length, and syntax issues.

- **Bandit**: Your "app's security guard" for Python, identifying **common security vulnerabilities**. It detects dangerous functions and insecure coding patterns.

Deliverables

You will use Pylint, Bandit, and Flake8 to analyze a provided Python program. Your task is to identify, document, and fix a minimum of four issues found by these tools, prioritizing high and medium severity findings. Finally, you will reflect on your experience with static analysis.

1. A **cleaned and updated version with** at least four issues fixed.
2. A **filled-out table** documenting the identified issues and how they were addressed.
3. **Answers to reflection questions** provided in the lab.

Steps:

Step 1:Inspect the Python File in the assigned repo:

- Open the .py file in the repo.
- Familiarize yourself with the code's structure and functionality.
- Run the program from the terminal.
    - python your_python_script.py

Step 2: Run Static Analysis Tools

Now that your environment is set up, run each analysis tool against the python script. The output will be redirected to text files for easier review.

1. **Run Pylint:** pylint your_python_script.py > pylint_report.txt
2. **Run Bandit:** bandit -r your_python_script.py > bandit_report.txt
3. **Run Flake8:** flake8 your_python_script.py > flake8_report.txt
4. You should now see pylint_report.txt, bandit_report.txt, and flake8_report.txt in your file explorer.
5. Open them to view the reports.

Step 3: Identify and Document Issues

Review the generated reports and cross-reference them with the common issues outlined for each tool. Use the provided table structure to document your findings. You may need to refer to the official documentation for Pylint, Bandit, and Flake8 if you need more context on specific error codes or warnings.

Example Known Issue Table:

Issue	Type	Line(s)	Description	Fix Approach
Mutable default arg	Bug	12	`logs=[]` shared across calls	Change default to `None` and initialize in method
... (Your findings)	...	...	...	...

Step 4: Fix Issues (30 minutes)

Based on your documented issues, begin fixing the problems in `your_python_script.py`.

1. **Handle all the issues reported from Bandit.**
2. **Get the score of Pylint above 9.5 after changes.**
3. **Handle all the formatting issues raised by Flake8.**
4. **Verify Fixes:** After making changes, **rerun the static analysis tools** and check the new reports (`pylint_report.txt`, `bandit_report.txt`, `flake8_report.txt`) to confirm that the issues you've addressed no longer appear.
5. Repeat this process until your fixes are verified.

Deliverables:

1. 3 Commits for changes you made for fixing the issues raised by three tools.
2. `pylint_report.txt`, `bandit_report.txt`, `flake8_report.txt` **(before and after)**